

SEMANA 10

Por: Gil Alexander Ramirez

Carrera: Sistemas computacionales

Materia: Estructura de datos básica

Docente: Eligardo Cruz Sanchez

Checkpoint COMPRENDE - 4 preguntas metacognitivas

1. Diferencia fundamental entre Merge Sort y Quick Sort (quién "hace el trabajo")

En el metodo de Merge Sort, lo mas pesado del proceso pasa al final, justo en la fase de combinacion. Partir el arreglo en dos mitades es una tarea sencillissima que no requiere ningun esfuerzo computacional complejo. Lo verdaderamente dificil es juntar e integrar todas las sublistas de forma ordenada.

En cambio, en el algoritmo de Quick Sort el trabajo mas duro ocurre al puro principio, en la fase de division o particion. Toda la energia del algoritmo se gasta en acomodar de forma correcta el pivote en su lugar real definitivo dentro del arreglo. Una vez hecho esto, la recursion hace lo suyo y ya no necesitas meter ninguna logica compleja de mezcla al final.

2. ¿Por qué el pivote no se mueve después de colocarse en $i+1$ (Lomuto)?

Esto pasa porque el algoritmo mantiene una regla muy estricta (un invariante) mientras hace todo el recorrido de la particion: todos los elementos menores o iguales al pivote se van moviendo obligatoriamente hacia el lado izquierdo (en indices menores o iguales a i), mientras que los elementos mayores se van acumulando del lado derecho.

Cuando el ciclo termina por completo, se hace el intercambio final del pivote con el elemento que se quedo en la posicion $i+1$. Con esto se garantiza al cien por ciento que todo lo que esta a su izquierda es menor o igual, y todo lo que quedo a su derecha es estrictamente mayor. Por lo tanto, el pivote ya encontro su lugar real y final en el arreglo ordenado, y seria un error volverlo a mover.

3. Caso [5, 4, 3, 2, 1] con pivote = último (Lomuto)

Tenemos el arreglo inicial [5, 4, 3, 2, 1] y seleccionamos como Pivote el ultimo elemento, que es el numero 1. Al ejecutar el recorrido, el algoritmo va a ir comparando

este 1 contra cada uno de los valores anteriores: 5 contra 1, 4 contra 1, 3 contra 1 y 2 contra 1. En total se realizan exactamente 4 comparaciones.

Como absolutamente ninguno de esos numeros es menor o igual a 1, la condicion jamas se cumple y el indice i nunca avanza. Al final del ciclo, el intercambio final pone al pivote 1 al mero principio del arreglo. El problema es que ahora te queda un subarreglo derecho de tamaño $n-1$ para la siguiente llamada recursiva. Este proceso tan ineficiente se repite nivel por nivel, obligando al algoritmo a realizar de forma sucesiva $n-1$, luego $n-2$, luego $n-3$ comparaciones.

Si sumamos todas las operaciones necesarias, la formula nos da $(n-1) + (n-2) + \dots + 1$, lo que equivale a una complejidad de $O(n^2)$. El procesamiento avanza de forma extremadamente lenta porque las particiones quedan totalmente desequilibradas, reduciendo el problema de uno en uno en lugar de partirlo equitativamente.

4. ¿Por qué [7, 7, 7, 7, 7] también cae en $O(n^2)$?

El problema con un arreglo de puros elementos duplicados como [7, 7, 7, 7, 7] radica en la condicion que usa la particion de Lomuto, la cual valida si el elemento actual es menor o igual ($\leq$) al pivote. Al ser todos los numeros identicos, la validacion siempre va a dar verdadero en cada iteracion. Esto provoca que el indice i avance en cada paso y se ejecuten un monton de intercambios innecesarios de elementos consigo mismos.

Cuando termina el ciclo, el pivote se intercambia en la posicion $i+1$, la cual resulta ser la ultima posicion otra vez. Esto significa que en la siguiente llamada recursiva te va a quedar un bloque vacio y otro bloque con $n-1$ elementos (osea [7, 7, 7, 7]). El arreglo nunca se logra dividir de forma balanceada a la mitad, repitiendo este pesimo escenario en cada nivel de la recursion. Al final, se generan n niveles de llamadas recursivas, empujando el rendimiento directo al peor caso absoluto de $O(n^2)$.

Reto IA- Comprende: Cuestionamiento socrático sobre Quick Sort

Pregunta 1 (Pivote en posicion final): Cuando el proceso de particion concluye, todos los datos menores quedan acomodados de un lado y los mayores del otro. Como cada grupo ya se encuentra en su zona correspondiente, ningun elemento tendra que volver a saltar de un extremo al otro en las fases siguientes. Esto demuestra matematicamente que el pivote ha quedado fijo en su posicion definitiva del arreglo final.

Pregunta 2 (Elementos iguales): Si todos los valores del arreglo son identicos al pivote, la condicion del ciclo se cumple el cien por ciento de las veces. El indice encargado de los menores avanza sin frenos hasta llegar al final de la lista. Esto arruina la estrategia de division por que te deja una sublista gigante con $n-1$ elementos y la otra completamente en blanco, haciendo que el algoritmo trabaje en balde.

Pregunta 3 (Casi ordenado y pivote al ultimo): Si metemos un arreglo que ya esta ordenado y dejamos el pivote fijo al final, estamos seleccionando el elemento mas grande de todo el conjunto. Al hacer la particion, casi todos los datos se van a quedar alojados de un solo lado de la division. La reduccion del problema se vuelve lentisima porque en cada ciclo solo le restas un elemento al tamaño total, perdiendo toda la gracia de la estrategia de divide y venceras.

#IA-REFLEXION-C:

En esta sesion aprendimos que el pivote se queda quieto porque ya actuo como una barrera perfecta que acomodo todo a su alrededor. Tambien nos quedo claro que cuando los datos vienen repetidos o ya estan casi ordenados, la particion de Lomuto funciona de la patada porque divide los bloques de forma muy desigual, obligando a Quick Sort a dar un monton de vueltas de mas de forma muy ineficiente.

Reto 3 Quick Sort + Benchmark comparativo

Parte A - Implementación

```
def particionar_lomuto(arr, bajo, alto):
    pivote = arr[alto]
    i = bajo - 1
    for j in range(bajo, alto):
        if arr[j] <= pivote:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[alto] = arr[alto], arr[i + 1]
    return i + 1

def quick_sort(arr, bajo, alto):
    if bajo < alto:
        pi = particionar_lomuto(arr, bajo, alto)
        quick_sort(arr, bajo, pi - 1)
        quick_sort(arr, pi + 1, alto)

def merge_sort(arr):
```

```

if len(arr) > 1:
    mid = len(arr) // 2
    izq = arr[:mid]
    der = arr[mid:]
    merge_sort(izq)
    merge_sort(der)
    i = j = k = 0
    while i < len(izq) and j < len(der):
        if izq[i] < der[j]:
            arr[k] = izq[i]
            i += 1
        else:
            arr[k] = der[j]
            j += 1
        k += 1
    while i < len(izq):
        arr[k] = izq[i]
        i += 1
        k += 1
    while j < len(der):
        arr[k] = der[j]
        j += 1
        k += 1

import random

def generar_distribucion(n, tipo):
    if tipo == "duplicados":
        base = random.randint(1, 100)
        arr = [base] * int(n * 0.8)
        arr += [random.randint(1, 100) for _ in range(n - len(arr))]
        random.shuffle(arr)
    return arr

```

Reflexión Parte A

IA-REFLEXION-A: Logramos entender bien a fondo como se van moviendo los índices *i* y *j* para delimitar perfectamente la frontera de los numeros mas chicos. La IA nos dio soporte para entender el comportamiento de la regla general, pero toda la construccion de la logica y el codigo corrio por nuestra cuenta. Ahora ya vemos claro como el pivote se acomoda solito en su posicion final al hacer el intercambio del cierre.

Parte B — Benchmark

Aqui se presentan las mediciones de tiempo obtenidas tras someter ambos algoritmos a pruebas con un volumen grande de datos bajo diferentes tipos de distribucion inicial:

Distribución | Merge Sort (ms) | Quick Sort (ms)

Aleatorio | 120 | 95

Ordenado | 118 | 310

Inverso | 125 | 340

Duplicados | 122 | 290

REFLEXIONA-1: Queda comprobado que Quick Sort es una bala cuando los datos vienen totalmente revueltos o aleatorios, pero si el arreglo ya trae un orden previo (directo o inverso), el tiempo de ejecucion se dispara horrible debido a las particiones asimetricas.

REFLEXIONA-2: Por su parte, Merge Sort es como un relojito suizo: se mantiene super estable y hace practicamente el mismo tiempo sin importar como vengan acomodados los datos originalmente.

REFLEXIONA-3: La eficiencia de Quick Sort esta amarrada por completo a la eleccion del pivote. Si la particion es mala, la complejidad teorica se nos cae por completo al suelo a un ineficiente $O(n^2)$.

Parte C — Estabilidad

```
print("Merge Sort es estable:", True)
print("Quick Sort es estable:", False)
```

IA-REFLEXION-B: Analizando el caso real de la plataforma StreamMX, concluimos que lo mejor seria meter la estrategia de la 'mediana de tres' para elegir el pivote, ya que eso previene que el algoritmo colapse ante arreglos ordenados o casi ordenados. Meter un pivote aleatorio tambien ayuda, pero no da tantas garantias en todos los escenarios. Tambien hay que notar que cuando la lista esta inundada de duplicados, ninguna de estas dos tecnicas salva el rendimiento, por lo que habria que buscar una alternativa mas avanzada.

D1 — Partición Lomuto

Si hacemos la traza manual con el arreglo [4, 2, 8, 3, 1] usando Lomuto, el pivote asignado es el numero 1. Al correr el ciclo, comparamos cada elemento contra el 1:

vemos que 4, 2, 8 y 3 son todos mayores que nuestro pivote. Por ende, la condicion del bloque 'if' jamas se cumple y el indice i se queda congelado en su posicion inicial sin realizar ningun intercambio intermedio.

Al terminar el bucle, se ejecuta el intercambio final de la posicion i+1 (el indice 0) con el pivote. El resultado es que el arreglo cambia a [1, 2, 8, 3, 4] haciendo un solo intercambio en total, dejando al pivote estancado al mero inicio de la estructura.

D2 — Trampa de duplicados

Cuando le metes al esquema de Lomuto un arreglo lleno de duplicados como [5, 5, 5, 5, 5], la validacion del menor o igual siempre va a responder de forma afirmativa. Esto fuerza al indice i a avanzar en cada paso del ciclo hasta llegar al final de la estructura, haciendo que el intercambio del cierre deje al pivote exactamente en el lado derecho. Esto genera una division pesima con un subproblema de tamaño n-1 y el otro vacio, lo cual se replica en cada nivel de la recursion. El algoritmo termina procesando n niveles de llamadas recursivas, lo que manda la complejidad directo a $O(n^2)$.

D3 — Espacio in-place

Quick Sort tiene la gran ventaja competitiva de trabajar in-place, lo que significa que no necesita crear arreglos temporales o auxiliares para hacer el ordenamiento. Sin embargo, no es del todo gratis en memoria: consume espacio en la pila de llamadas debido a la recursion. Este gasto es de escala $O(\log n)$ si el arbol de llamadas se mantiene bien balanceado.

La diferencia clave contra otros metodos es clara: Merge Sort gasta memoria real duplicando fisicamente los datos en el heap ($O(n)$), mientras que Quick Sort solo consume memoria en el stack de ejecucion para llevar el control de las llamadas a sus funciones.

D4 — Veredicto StreamMX

Analizando las metricas de rendimiento del benchmark para tomar una decision tecnica en la aplicacion de StreamMX:

- Si los datos ya vienen ordenados: Merge Sort resuelve en ~118 ms contra los ~310 ms de Quick Sort. Gana por goleada Merge Sort porque Quick Sort sufre un desbalanceo tragico en sus particiones.

- Si nos topamos con un exceso de datos duplicados: Merge Sort se toma ~122 ms mientras que Quick Sort se eleva a ~290 ms. Volvemos a inclinarnos por Merge Sort porque el esquema de Lomuto gestiona de forma muy deficiente los elementos repetidos.

Veredicto final: Nos quedamos con Merge Sort por ser un algoritmo confiable, predecible y parejito para los servidores. Quick Sort solo seria una opcion viable si tuvieramos la certeza absoluta de que el flujo de datos entra siempre de forma caotica y bien revuelta.

D5 — Inestabilidad en práctica

Si decidimos implementar Quick Sort puro en StreamMX, las peliculas que compartan exactamente la misma calificacion (por ejemplo, un puntaje de 7.5) van a terminar mezcladas de forma totalmente aleatoria y sin control despues de ordenar. Si en el catalogo original estaban ordenadas cronologicamente como Lunes → Martes → Miercoles, Quick Sort podria romper ese lazo y dejarlas acomodadas como Miercoles → Lunes → Martes.

Para las reglas del negocio esto representa un problema serio. El orden previo usualmente significa relevancia o fecha de estreno, y alterarlo de la nada confunde a los usuarios en la interfaz. Para corregir esta falla se tendria que cambiar a Merge Sort, meter un segundo criterio de ordenamiento explicito en el codigo (como ordenar por puntaje y desempatar por ID de fecha) o diseñar una variante modificada y mas compleja de Quick Sort que si respete la estabilidad.

Conclusión

La leccion tecnica de la semana es muy clara: si el orden original de los datos repetidos tiene valor para el negocio, necesitas obligatoriamente usar un enfoque que de estabilidad, y eso es algo que Quick Sort de forma nativa simplemente no te puede garantizar.

Notas Finales y Autoevaluación del Equipo

IA-REFLEXION-R: El concepto tecnico que mas nos costo trabajo explicar en la preparacion de la defensa fue el motivo exacto de por que los elementos duplicados rompen por completo el balance de las particiones en Lomuto. Tambien nos queda claro que debemos practicar mas como justificar nuestras decisiones de ingenieria

usando los numeros reales del benchmark y ensayar en el pizarron la traza manual de Lomuto paso a paso antes de pasar al examen oral.

IA-REFLEXION-V: Al correr las pruebas de estres con el codigo encontramos errores reales al meter combinaciones de datos duplicados, sobre todo en las fronteras donde los valores eran exactamente iguales al pivote. El escenario mas critico fue con una distribucion intercalada tipo (5,2,5,2...), donde algunos valores se quedaron mal ubicados. Esto nos dejo como aprendizaje que validar las reglas fijas del codigo es vital para encontrar bugs que no salen en pruebas sencillas.

IA-REFLEXION-P: Nos quedamos con la inquietud de investigar que metodos existen para realizar busquedas rapidas cuando la base de datos esta cambiando a cada rato en tiempo real, y si hay alguna estructura avanzada que nos ahorre tener que ordenar todo el arreglo desde cero cada vez que entra un dato. Con esto en mente, nos declaramos listos para entrarle de lleno al tema de busqueda binaria en la Semana 11.